

Proyecto 1. Propuesta de Caso de Uso

Redes de Computadoras - CC3067

José Antonio Mérida Castejón - 201105

3 de agosto de 2026

Introducción

Se propone implementar un servidor MCP que funcione como un **catálogo de datos interno con validación de calidad**, dirigido al equipo de analítica de una empresa mediana.

El problema que atiende es común en organizaciones que acumulan datasets provenientes de distintas fuentes (exportaciones de CRM, ventas, recursos humanos) sin documentación centralizada. Los analistas invierten tiempo considerable en determinar qué datos existen, qué significa cada columna y en qué unidades está expresada, y si un extracto recibido cumple con las condiciones mínimas de calidad antes de utilizarlo en un modelo.

El servidor expone herramientas de solo lectura que permiten al analista formular estas preguntas en lenguaje natural a través del chatbot anfitrión. El LLM coordina las llamadas a las herramientas e interpreta los resultados, pero no genera información por su cuenta.

El patrón corresponde a herramientas de uso industrial establecido, como los archivos `schema.yml` de dbt, las suites de expectativas de Great Expectations, y catálogos de datos como DataHub o Amundsen. Cabe aclarar que la propuesta *no* constituye un *feature store* ya que no calcula ni sirve valores de *features* para inferencia.

Origen de la información

El servidor obtiene la información de dos fuentes complementarias, ambas locales al servidor:

1. **Archivos de datos** en formato Parquet, sobre los cuales se calculan todas las estadísticas y validaciones mediante `pandas`. Se utilizarán cinco datasets públicos de dominio empresarial:
 - *Telco Customer Churn* (IBM / Kaggle) — suscripciones y abandono de clientes.
 - *Superstore Sales* (Kaggle) — órdenes, regiones y rentabilidad.
 - *Online Retail II* (UCI) — transacciones de comercio electrónico.
 - *HR Employee Attrition* (IBM / Kaggle) — rotación de personal.
 - *Extracto mensual sin validar* — copia de *Telco Customer Churn* con inconsistencias introducidas de forma controlada y reproducible mediante un script con semilla fija, para demostrar la detección de problemas de calidad.

2. **Catálogo declarativo** en un archivo YAML (`catalog.yaml`) escrito manualmente, que actúa simultáneamente como diccionario de datos y como definición de reglas de calidad. Para cada columna se declara su tipo, descripción, unidad y las reglas que debe cumplir.

Ejemplo de definición en el catálogo

```
- name: tenure_months
  type: integer
  description: Meses desde la primera compra del cliente
  unit: months
  rules:
    - not_null
    - range: [0, 600]
      severity: error
```

Funcionalidad expuesta

El servidor expone cuatro herramientas principales y tres complementarias, todas de solo lectura y sin estado, lo cual permite desplegar la misma implementación de forma local y remota sin modificaciones.

Herramienta	Descripción
<code>list_datasets</code>	Lista los datasets registrados con su descripción, origen, número de filas y fecha de actualización.
<code>describe_dataset</code>	Devuelve el diccionario de datos de un dataset: columnas, tipos, descripciones, unidades y reglas declaradas.
<code>profile_column</code>	Calcula estadísticas sobre una columna según su tipo: cuantiles y valores atípicos para numéricas, cardinalidad y valores frecuentes para categóricas, rango y vacíos para fechas.
<code>validate_dataset</code>	Ejecuta las reglas declaradas contra los datos reales y devuelve un reporte de calidad con las violaciones encontradas.

Herramienta	Descripción
<code>search_catalog</code>	Búsqueda semántica sobre el catálogo mediante <i>embeddings</i> . Consulta dos índices independientes (datasets y columnas) y devuelve ambos conjuntos de resultados por separado.
<code>chart_column</code>	Genera una gráfica (histograma, barras o serie de tiempo) y la devuelve como bloque de contenido tipo imagen del protocolo MCP.
<code>find_undocumented_columns</code>	Compara las columnas presentes en el archivo contra las declaradas en el catálogo, para detectar documentación desactualizada.

Búsqueda semántica en dos niveles

La herramienta `search_catalog` construye al iniciar el servidor dos índices vectoriales a partir de los metadatos del catálogo: uno de datasets, que codifica nombre, descripción y origen, y responde a consultas de granularidad amplia; y otro de columnas, que codifica el nombre y descripción de cada columna junto con la descripción de su dataset, dado que el nombre de una columna por sí solo suele ser demasiado breve para producir una representación vectorial informativa. La recuperación se realiza por similitud coseno mediante multiplicación matricial con `numpy`; dado el tamaño reducido del índice (≈ 100 vectores), una búsqueda exhaustiva es la solución apropiada y no se justifica el uso de una base de datos vectorial.

Los resultados de ambos índices se devuelven en conjuntos separados y no en una única lista ordenada, ya que sus puntajes no son directamente comparables entre sí. Si el servicio de *embeddings* no se encuentra disponible, la herramienta recurre a una búsqueda por coincidencia de palabra clave sobre los mismos campos, de modo que la funcionalidad se degrada pero no falla.

Reporte de calidad de datos

La herramienta `validate_dataset` ejecuta contra los datos reales las reglas declaradas en el catálogo, que operan a nivel de columna (`not_null`, `unique`, `range`, `allowed_values`, `regex`, `type_check`) o a nivel de dataset (`row_count`, `freshness`). Cada regla se declara con una severidad, `error` invalida el dataset para su uso posterior, mientras que `warning` señala una anomalía que amerita revisión sin bloquear el flujo de trabajo.

El reporte devuelto incluye un veredicto general con el conteo de reglas evaluadas, aprobadas y fallidas; el detalle de cada verificación con su severidad y el número y porcentaje de violaciones; y hasta cinco valores de ejemplo por violación, de modo que el analista pueda localizar el problema en el archivo original. Para demostrar la herramienta, el dataset *Extracto mensual sin validar* contiene cinco inconsistencias introducidas deliberadamente, cada una dirigida a un tipo de regla distinto.

Privacidad y servicios externos

La generación de *embeddings* se delega a un servicio externo, lo cual evita incorporar un modelo local al contenedor y mantiene una imagen ligera con tiempos de arranque en frío reducidos. El

servidor transmite a dicho servicio **únicamente metadatos**: nombres de dataset, nombres de columna, descripciones y unidades, todos redactados manualmente en el archivo de catálogo. En ningún momento se transmiten registros ni valores de los archivos de datos, aunque sí las consultas del usuario, al ser necesario codificarlas para la búsqueda. Esta separación entre metadatos y datos es una decisión de diseño deliberada, y es lo que hace viable el caso de uso en un entorno empresarial donde los datos suelen estar sujetos a restricciones contractuales o regulatorias.

Escenario de demostración

1. *¿Qué datasets tenemos disponibles?* → `list_datasets`
2. *¿Qué contiene `customer_churn`?* → `describe_dataset`
3. *¿Está limpio el extracto de este mes?* → `validate_dataset` reporta las cinco inconsistencias, diferenciando errores de advertencias.
4. *¿Cómo se distribuyen los cargos mensuales?* → `profile_column` y `chart_column`
5. *¿Tenemos algo sobre cuánto tiempo llevan las personas con nosotros?* → `search_catalog` recupera `tenure_months` del dataset de clientes y `YearsAtCompany` del de personal, pese a no existir coincidencia literal de palabras.